Capstone Project 3: RAG Pipeline with Azure Prompt Flow & Azure AI Services

---

Objective

The objective of this capstone project is to design and implement a production-ready Retrieval-Augmented Generation (RAG) pipeline using **Azure Prompt Flow** within **Azure Machine Learning Studio**. The project will cover:

- Building a modular RAG pipeline using Azure Prompt Flow's visual canvas
- Integrating Azure AI Search as a vector store for efficient document retrieval
- Using Azure OpenAI for embeddings and response generation
- Evaluating the RAG pipeline using built-in Prompt Flow evaluation flows
- Deploying the system as a managed REST API endpoint on Azure

---

Dataset

You may use any text corpus relevant to your chosen domain (e.g., financial documents, medical articles, legal contracts, or product manuals).

- **Knowledge Base**: Chunked and vectorized document embeddings stored in **Azure AI Search**.
- **Input**: User queries in natural language.
- **Output**: Generated responses augmented with retrieved document context, grounded in your knowledge base.

Problem Statement

Retrieval-Augmented Generation (RAG) systems are critical for building trustworthy AI applications that combine large language models with external knowledge bases. However, most RAG implementations lack structured, visual, and production-ready pipelines that allow retrieval, grounding, evaluation, and deployment in an enterprise-grade manner.

You are tasked with building a complete RAG pipeline on **Azure Prompt Flow** that:

- Indexes a domain-specific document corpus into **Azure AI Search** as a vector index
- Retrieves relevant context from the vector index using hybrid search (semantic + keyword)
- Embeds user queries using **Azure OpenAI text-embedding-ada-002**
- Generates grounded responses using **Azure OpenAI GPT-4.1-mini** conditioned on retrieved context
- Implements a structured prompt template with context injection and conversation history
- Evaluates the pipeline on metrics such as Groundedness, Relevance, and Fluency
- Deploys the flow as a managed **REST API endpoint** on Azure Machine Learning

The system should support domain-specific use cases across three industries:

To simulate real-world applications, the capstone integrates RAG as a core pipeline across three industries:

1. Financial Market Intelligence (Student ID 1–9)

- **Knowledge Base**: Tesla 10-K and 10-Q filings, market reports, earnings call transcripts
- **Query Examples**: Revenue by quarter, R&D expenditure trends, debt-to-equity ratios
- **Expected Output**: Factual answers grounded in the uploaded financial filings with fiscal year and quarter references

2. Healthcare Knowledge Assistant (Student ID 10–18)

- **Knowledge Base**: Medical guidelines, clinical research articles, drug reference documents
- **Query Examples**: Treatment protocols, drug interactions, diagnostic criteria
- **Expected Output**: Clinically grounded answers citing the specific guideline or article retrieved

3. Enterprise Customer Support Automation (Student ID 19–27)

- **Knowledge Base**: Product manuals, FAQ documents, support policy documents
- **Query Examples**: Return policies, troubleshooting steps, warranty terms
- **Expected Output**: Accurate support answers grounded in official company documentation

---

1: Azure Resource Setup & Environment Configuration

Capstone Tasks

1. Azure Resource Provisioning

- Create an **Azure Resource Group** to organize all project resources
- Provision an **Azure OpenAI** resource and deploy two models:

  text-embedding-ada-002 for document and query embedding

  gpt-4.1-mini for response generation

- Provision an **Azure AI Search** resource to serve as the vector store
- Create an **Azure Machine Learning Workspace** where Prompt Flow will be built and managed

2. Connection Configuration in Prompt Flow

- Create an **Azure OpenAI Connection** in Prompt Flow storing the API key, endpoint URL, and API version securely
- Create an **Azure AI Search Connection** storing the Search endpoint URL and admin key
- Verify both connections are active and available to the flow

---

2: Document Indexing & Vector Store Creation

Capstone Tasks

1. Document Preparation

- Collect and organize domain-specific source documents (PDF, DOCX, or TXT format)
- Upload documents as a local folder or Azure ML Data Asset

2. Vector Index Creation Using Azure Prompt Flow

- Use the **Vector Index wizard** inside Prompt Flow to create a vector index:

- Monitor the indexing pipeline job to completion

- Verify the index is live with status **"Ready"** in Azure AI Search

---

Section 3: RAG Flow Development in Azure Prompt Flow

Capstone Tasks

1. Flow Setup

- Create a new **Standard Flow** in Azure Prompt Flow named rag-pipeline-flow
- Start a compute session (Serverless or Compute Instance)
- Define flow inputs:
  question (type: string) — the user's natural language query
  chat_history (type: list) — conversation history for multi-turn support

2. Query Embedding Node

- Add an **Embedding node** (embed_question) using the Azure OpenAI connection
- Map input question → ${inputs.question}

3. Document Retrieval Node

- Add a **Vector DB Lookup node** (retrieve_docs) connected to Azure AI Search

4. Context Formatting Node

- Add a **Python node** (format_context) to clean and format retrieved chunks
- Concatenate retrieved document chunks into a single readable context string
- Include source file references for traceability

5. Prompt Construction Node

- Add a **Prompt node** (build_prompt) with a Jinja2 template that:

    a. Injects the formatted context into the system message
    b. Includes conversation history for multi-turn awareness
    c. Passes the user question as the human turn
    d. Instructs the model to answer only from provided context

6. Answer Generation Node

- Add an **LLM node** (generate_answer) using Azure OpenAI GPT-4.1-mini
- Map input to the constructed prompt from the previous node

7. Flow Output Configuration

- Define flow outputs:
  answer (string): the generated response from the LLM node
  retrieved_sources (list): the raw retrieved chunks for traceability

---

Section 4: Testing & Evaluation of the RAG Flow

Capstone Tasks

1. Single Query Testing

- Use the **Chat button** inside the Prompt Flow canvas to test the flow interactively

- Run single test queries relevant to your domain knowledge base

- Inspect each node's output individually to verify:

    a. Embedding node returns a valid vector
    b. Retrieval node returns 5 relevant document chunks
    c. Context formatter returns clean concatenated text
    d. Prompt node constructs a well-formed grounded prompt
    e. LLM node returns an accurate, grounded answer

---

2. Batch Evaluation Dataset Preparation

- Create a .json evaluation file with at least 10 question-answer pairs:

Sample:

{"question": "What was Tesla's total revenue in FY2023?", "ground_truth": "Tesla's total revenue in FY2023 was $96.77 billion."}

{"question": "What is Tesla's gross profit margin in Q3 2023?", "ground_truth": "Tesla reported a gross margin of 17.9% in Q3 2023."}

3. Built-in Evaluation Flow

- In the flow canvas, click **"Evaluate"** → **"Built-in Evaluation"**
- Select the **QA Evaluation** template
- Map evaluation inputs:
  a. question → dataset column question
  b. answer → flow output answer
  c. context → flow output retrieved_sources
  d. ground_truth → dataset column ground_truth
- Configure the evaluator LLM to use your Azure OpenAI GPT-4o connection
- Submit the evaluation batch run

---

4. Evaluation Metrics Analysis

- Review scores for all four metrics produced by the evaluation flow:
  a. **Groundedness**: Is the answer supported by the retrieved context? (Score 1–5)
  b. **Relevance**: Does the answer address the question? (Score 1–5)
  c. **Fluency**: Is the answer grammatically correct and readable? (Score 1–5)
  d. **Similarity**: How close is the answer to the ground truth? (Score 1–5)
- Analyze low-scoring questions and identify whether the issue is in retrieval (wrong chunks returned) or generation (LLM ignored context)
- Tune chunk size, Top K, temperature, or prompt instructions based on findings and re-evaluate

---

5. Deployment as a Managed REST API Endpoint

Capstone Tasks

1. Deploy the Flow

- In the Prompt Flow canvas, click **"Deploy"**
- Configure the deployment:
  a. Endpoint name: rag-pipeline-endpoint
  b. Deployment name: rag-pipeline-v1
  c. Virtual machine: Standard_DS3_v2
  d. Instance count: 1
- Confirm that both Azure OpenAI and Azure AI Search connections are included
- Click **"Create"** and wait for endpoint status to show **"Healthy"**

---

2. Endpoint Testing

- Navigate to **Endpoints** in Azure ML Studio → select your endpoint → click **"Test"** tab
- Send a test payload:

```
{
  "question": "What was Tesla's net income in Q2 2023?",
```

"chat_history": []

}

- Verify the JSON response contains a grounded answer field

---

3. Consume the Endpoint

- Go to the "**Consume**" tab of the endpoint and copy:
  a. REST endpoint URL
  b. Primary authentication key
- Demonstrate calling the endpoint programmatically using Python requests:

```
import requests
response = requests.post(
    url="https://your-endpoint.eastus.inference.ml.azure.com/score",
    headers={
        "Content-Type": "application/json",
        "Authorization": "Bearer YOUR_PRIMARY_KEY"
    },
    json={
        "question": "What is Tesla's debt-to-equity ratio in FY2023?",
        "chat_history": []
    }
)
print(response.json()["answer"])
```